Sensor Signal Analysis for Predicting Semiconductor Manufacturing Defects

Paul J Anderson

Abstract

This project analyzes the SECOM manufacturing dataset, originally gathered for fault detection in semiconductor production. The dataset comprises 1,567 product observations collected at a specific stage in the manufacturing process. It contains 590 sensor measurements per product along with a binary quality indicator: -1 for passed units and 1 for failed units. Note that the data has been anonymized, and this analysis relies solely on numerical values without additional contextual metadata, which in real-world scenarios is essential for building meaningful models.

Project Objective: To identify sensor patterns linked to product failures and develop a predictive model that can accurately forecast faulty products.

Key Questions:

Explanatory: Which sensors are most indicative of product failure, and how do their readings vary between passed and failed units?

Predictive: Can we construct an accurate classification model—such as logistic regression—that predicts product failure based on critical sensor readings?

Obtain

We begin by importing the SECOM dataset from a CSV file downloaded from Kaggle:

```
secom <- read_csv("C:/Users/ander/OneDrive - Oregon State University/Classes/2025/Spring/ST518/M9/uci-se
```

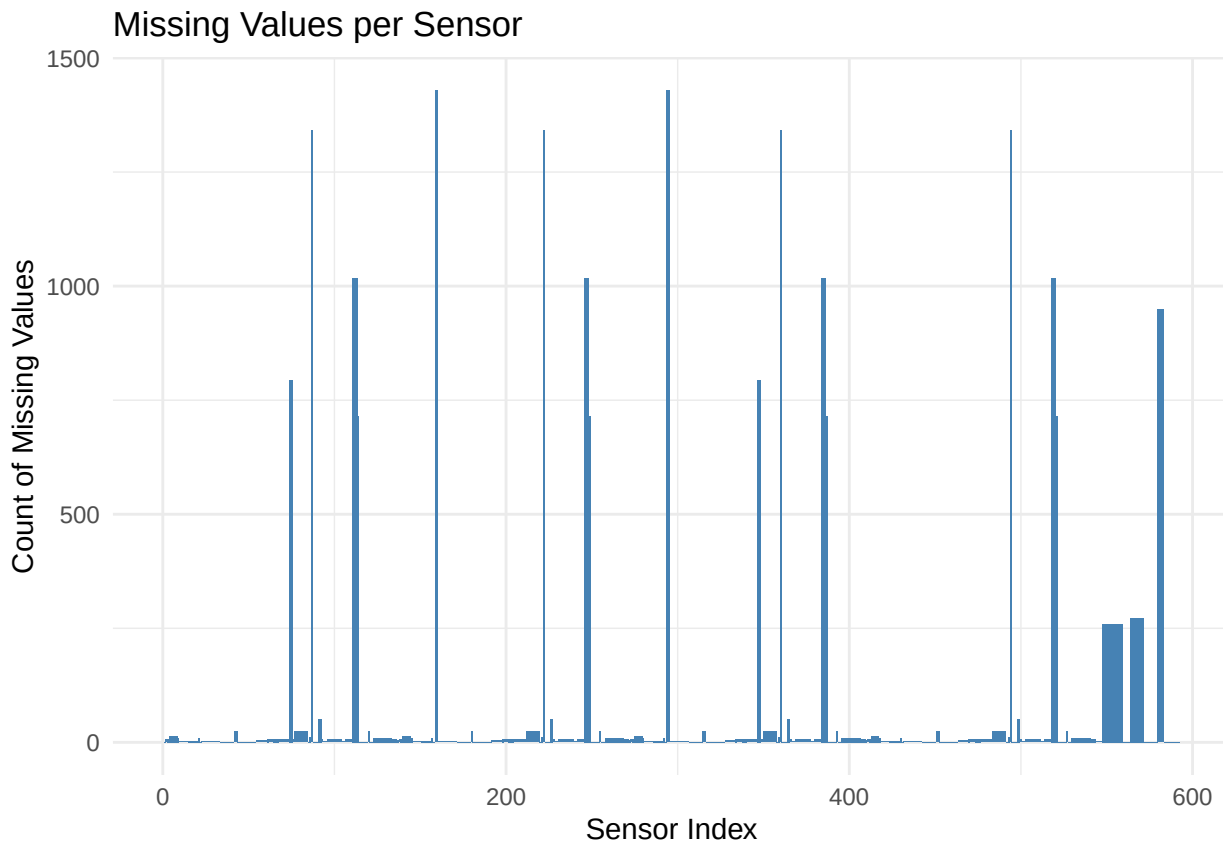
Scrub

In the scrubbing step, we clean the dataset by handling missing values and reducing noise. This involves removing sensors (columns) with a high proportion of missing data, dropping rows with any remaining missing values, recoding the target variable for clarity, and filtering out sensors with zero variance that provide no useful information. These actions improve data quality and ensure the dataset is suitable for reliable analysis and modeling.

We begin the data scrubbing process by identifying the number of missing values in each sensor column.

```
missing_counts <- colSums(is.na(secom))
missing_df <- data.frame(
  Sensor = seq_along(missing_counts),
  Missing = missing_counts
)

ggplot(missing_df, aes(x = Sensor, y = Missing)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(title = "Missing Values per Sensor",
```

```
       x = "Sensor Index",
       y = "Count of Missing Values") +
  theme_minimal()
```

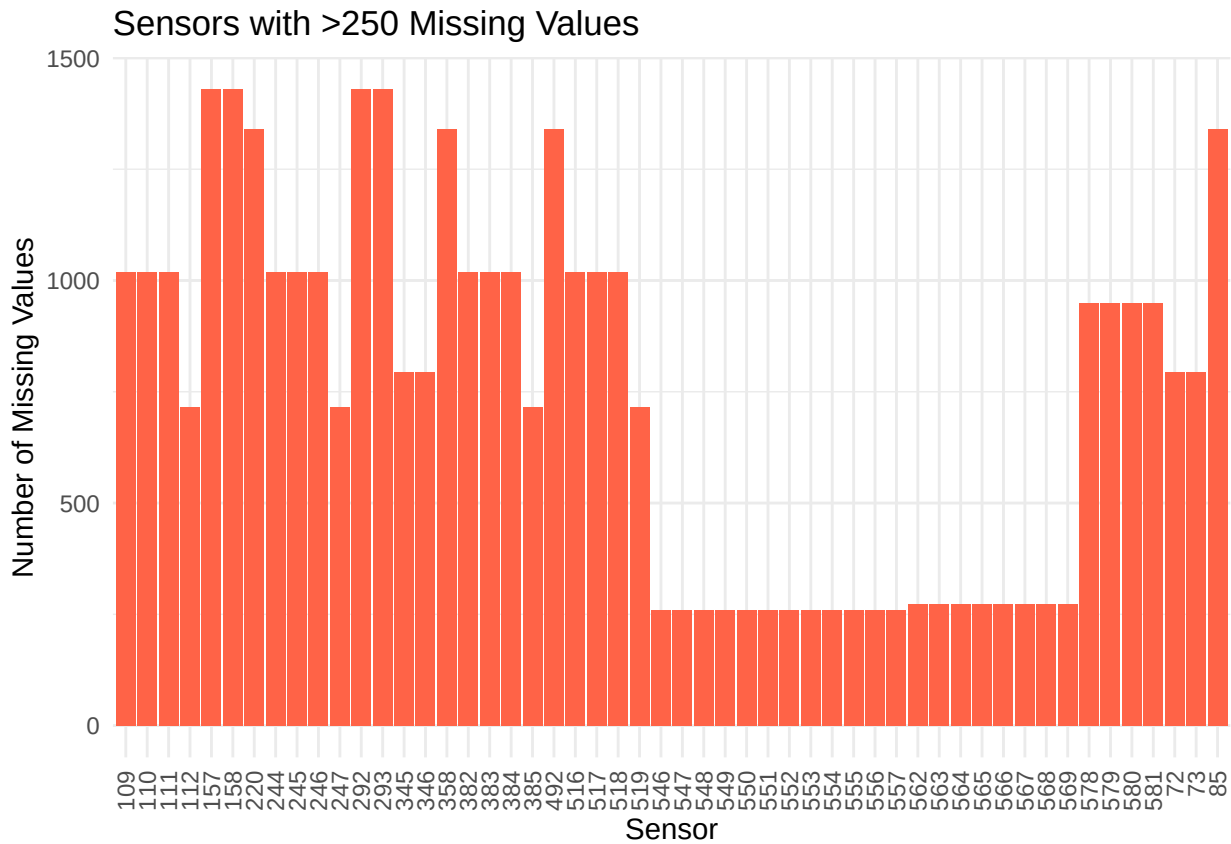


Next, we filter out sensors with a large amount of missing data (more than 150 missing values):

```
missing_over_150 <- missing_counts[missing_counts > 150]

# convert to data frame for viewing
missing_df <- data.frame(
  Sensor = names(missing_over_150),
  MissingValues = missing_over_150
)
```

We first visualize sensors with more than 250 missing values:

```
ggplot(missing_df, aes(x = Sensor, y = MissingValues)) +
  geom_bar(stat = "identity", fill = "tomato") +
  labs(title = "Sensors with >250 Missing Values",
       x = "Sensor",
       y = "Number of Missing Values") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```



This code cleans the secom dataset by removing columns with over 150 missing values, dropping rows with any remaining NAs, recoding the Pass/Fail variable from -1/1 to 0/1, renaming it to Label, and eliminating columns with zero variance. This prepares a cleaner, more informative dataset for analysis and modeling.

```
secom_clean <- secom %>%
  select(where(~ sum(is.na(.)) <= 150)) %>%          # Drop columns with >150 missing values
  drop_na() %>%                                      # Drop rows with any remaining missing values
  mutate(`Pass/Fail` = ifelse(`Pass/Fail` == -1, 0, 1)) %>%  # Convert -1 to 0, keep 1 as-is
  rename(Label = `Pass/Fail`) %>%                    # Rename Pass/Fail column to Label
  select(where(~ var(.x, na.rm = TRUE) != 0))        # Remove columns with zero variance
```

We present the dataset dimensions before and after cleaning to clearly illustrate the impact of the preprocessing steps. These steps include removing columns with excessive missing values, dropping rows containing remaining missing data, recoding the response variable, and eliminating sensors with zero variance.

Such preprocessing ensures the dataset is clean, informative, and well-prepared for subsequent modeling.

```
cat("Original rows:", nrow(secom), "\n")
```

```
## Original rows: 1567
```

```
cat("Original columns:", ncol(secom), "\n")
```

```
## Original columns: 592
```

```
cat("Rows after filtering:", nrow(secom_clean), "\n")
```

```
## Rows after filtering: 1393
```

```r
cat("Columns after filtering:", ncol(secom_clean), "\n")
```

```
## Columns after filtering: 424
```

Explore

In the explore step, we examine the cleaned dataset to understand its structure, distribution, and key characteristics. This includes summarizing statistics like variance and missingness, visualizing sensor readings, and identifying patterns or anomalies. We will also assess the relationship between sensor measurements and product outcomes, such as differences in sensor readings between passed and failed products. This exploration helps us gain insights, guide feature selection, and informs the modeling approach by highlighting important variables and potential data issues.

Here we pull the basic statistics for the first 10 variables, including measures such as minimum, first quartile, median, mean, third quartile, and maximum values. This summary provides an initial sense of the data's distribution and variability across these sensors. For example, the median variance near 1 suggests moderate spread in most sensors, but the higher mean and maximum values indicate some sensors have large outliers or skewed distributions. These insights help identify sensors that may require further scrutiny or preprocessing before modeling. We also note the date range in the dataset featured in Time from 2 am January 8th, 2008 to nearly 7pm December 10th of the same year.

```r
summary(secom_clean[1:10])
```

```
##       Time                              0              1              2
##  Min.   :2008-01-08 02:02:00.00   Min.   :2743   Min.   :2159   Min.   :2061
##  1st Qu.:2008-07-09 12:04:00.00   1st Qu.:2966   1st Qu.:2451   1st Qu.:2181
##  Median :2008-08-22 12:13:00.00   Median :3012   Median :2499   Median :2201
##  Mean   :2008-07-28 23:02:52.06   Mean   :3015   Mean   :2495   Mean   :2200
##  3rd Qu.:2008-09-20 18:56:00.00   3rd Qu.:3057   3rd Qu.:2539   3rd Qu.:2219
##  Max.   :2008-12-10 18:47:00.00   Max.   :3356   Max.   :2846   Max.   :2315
##        3              4                   6                7
##  Min.   :   0   Min.   :   0.6815   Min.   : 82.13   Min.   :0.0000
##  1st Qu.:1084   1st Qu.:   1.0160   1st Qu.: 98.13   1st Qu.:0.1211
##  Median :1285   Median :   1.3076   Median :101.67   Median :0.1224
##  Mean   :1384   Mean   :   2.9283   Mean   :101.23   Mean   :0.1222
##  3rd Qu.:1589   3rd Qu.:   1.5188   3rd Qu.:104.59   3rd Qu.:0.1238
##  Max.   :3715   Max.   :1114.5366   Max.   :129.25   Max.   :0.1286
##        8               9
##  Min.   :1.191   Min.   :-0.0534000
##  1st Qu.:1.413   1st Qu.:-0.0108000
##  Median :1.463   Median :-0.0013000
##  Mean   :1.465   Mean   :-0.0008607
##  3rd Qu.:1.519   3rd Qu.: 0.0085000
##  Max.   :1.656   Max.   : 0.0749000
```

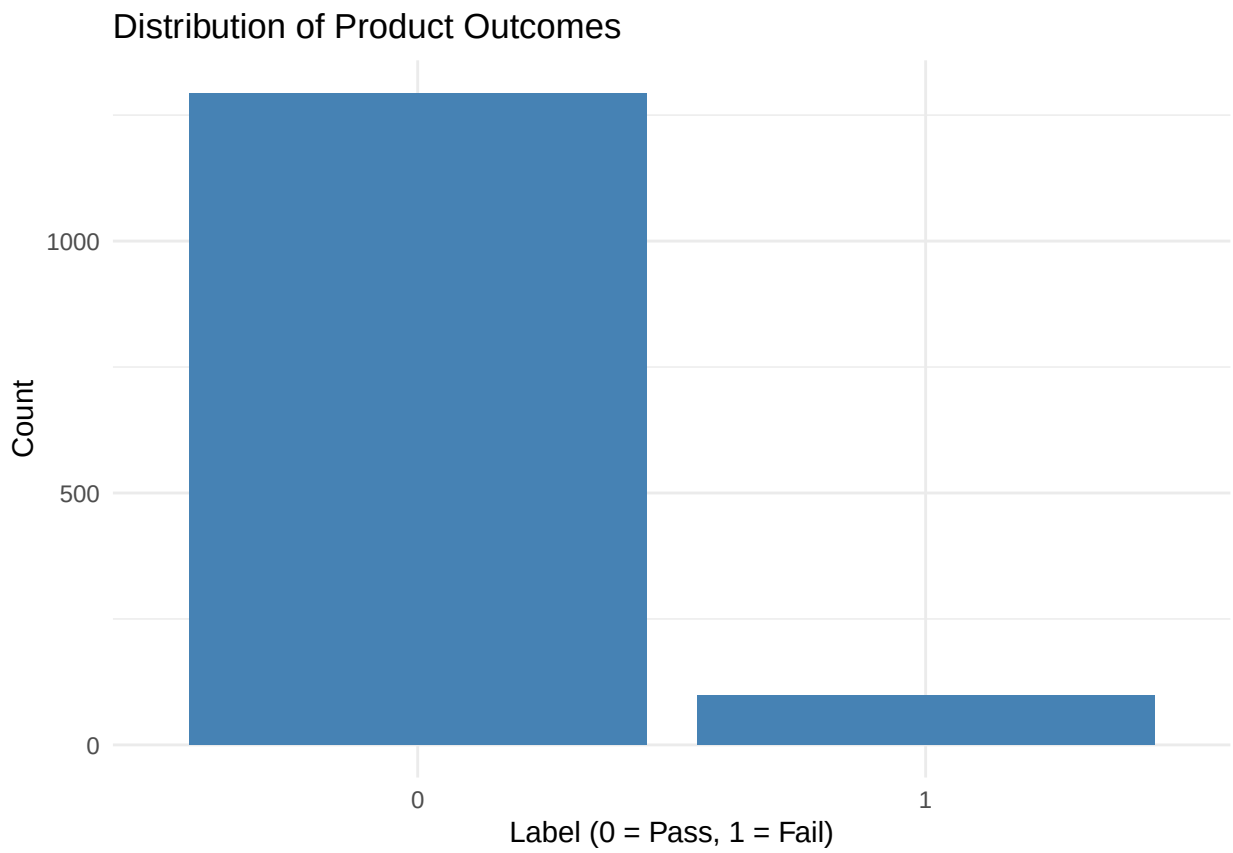
We plot the distribution of outcomes. Here we see that the dataset is imbalanced.

```r
table(secom_clean$Label)
```

```
##
##    0    1
## 1294   99
```

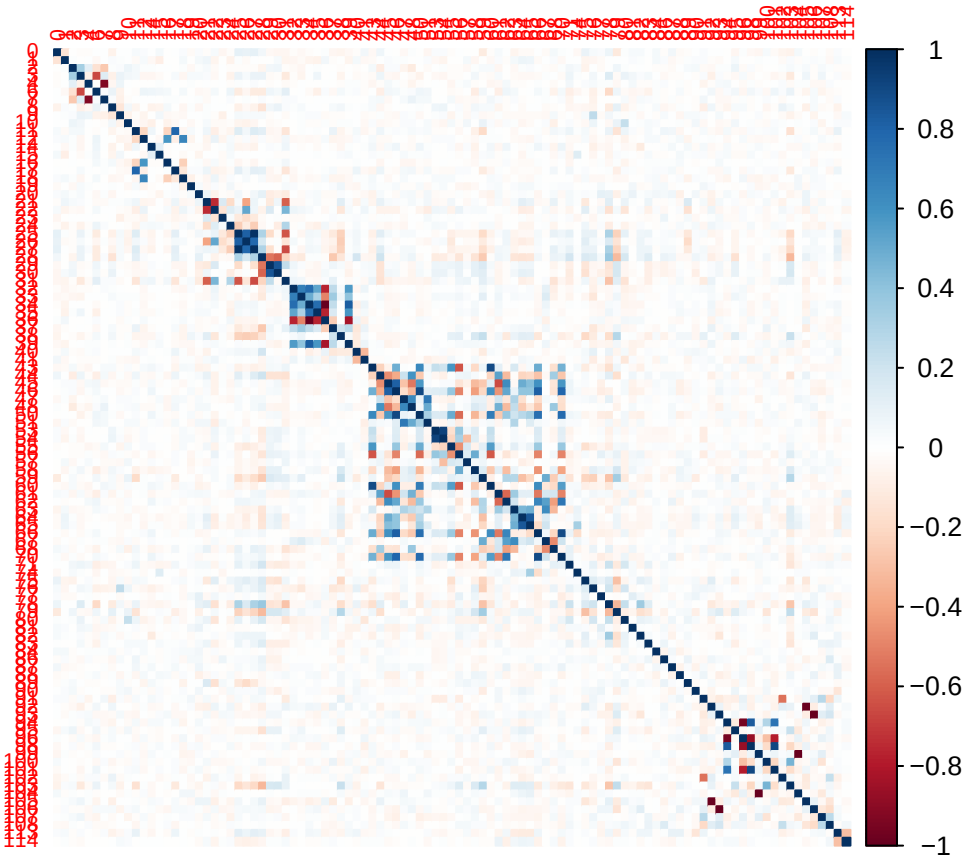
```r
ggplot(secom_clean, aes(x = factor(Label))) +
  geom_bar(fill = "steelblue") +
  labs(title = "Distribution of Product Outcomes",
```

```
       x = "Label (0 = Pass, 1 = Fail)",
       y = "Count") +
  theme_minimal()
```



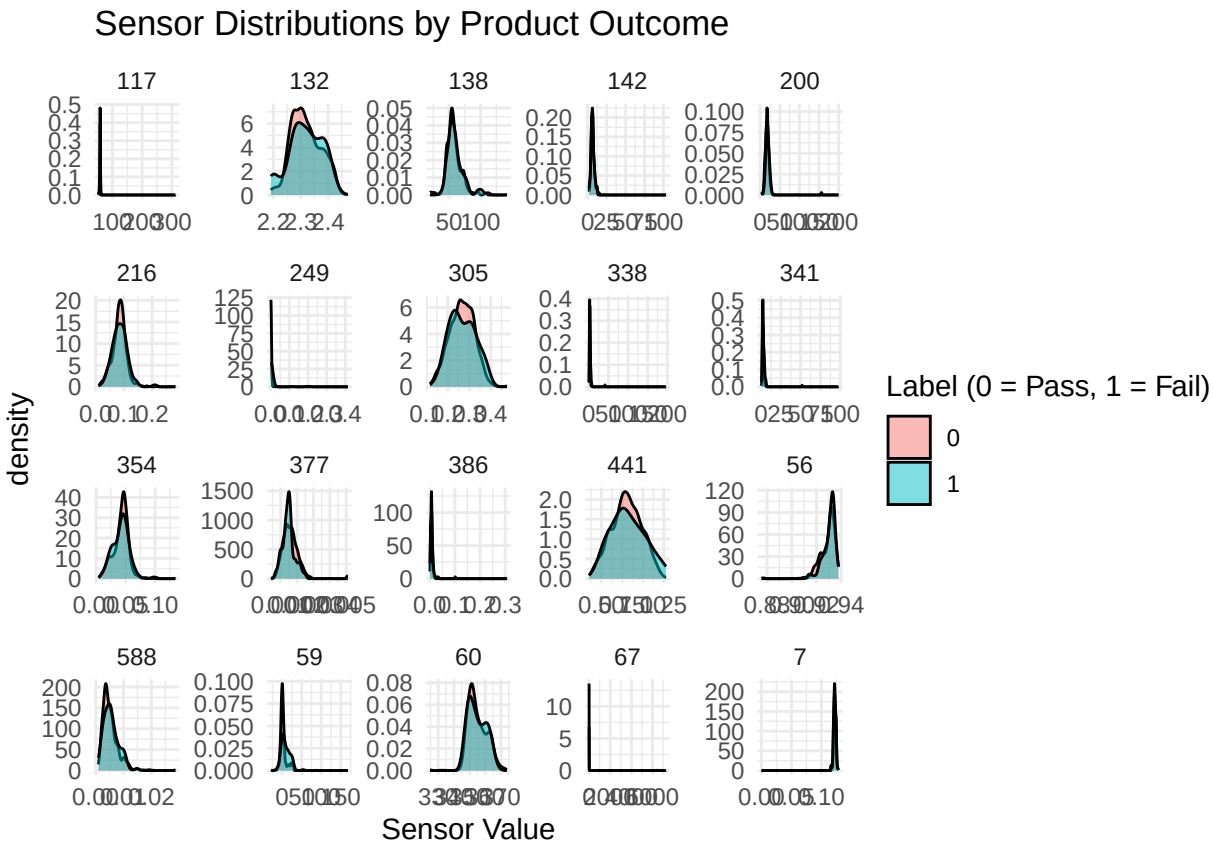
From our correlation matrix of the first 100 variables, we observed clusters of correlated sensors likely related to time or process stages. We explored approaches such as time-based splits, moving windows, and time-split AUC evaluation to address these temporal dependencies. However, given the course scope, we acknowledge these correlations as a limitation and proceed with modeling while noting that ignoring temporal structure may impact model performance and generalizability.

```
# compute correlations for a manageable subset of variables
cor_mat <- cor(secom_clean[, 2:102])  # adjust as needed

library(corrplot)
corrplot(cor_mat, method = "color", tl.cex = 0.7)
```



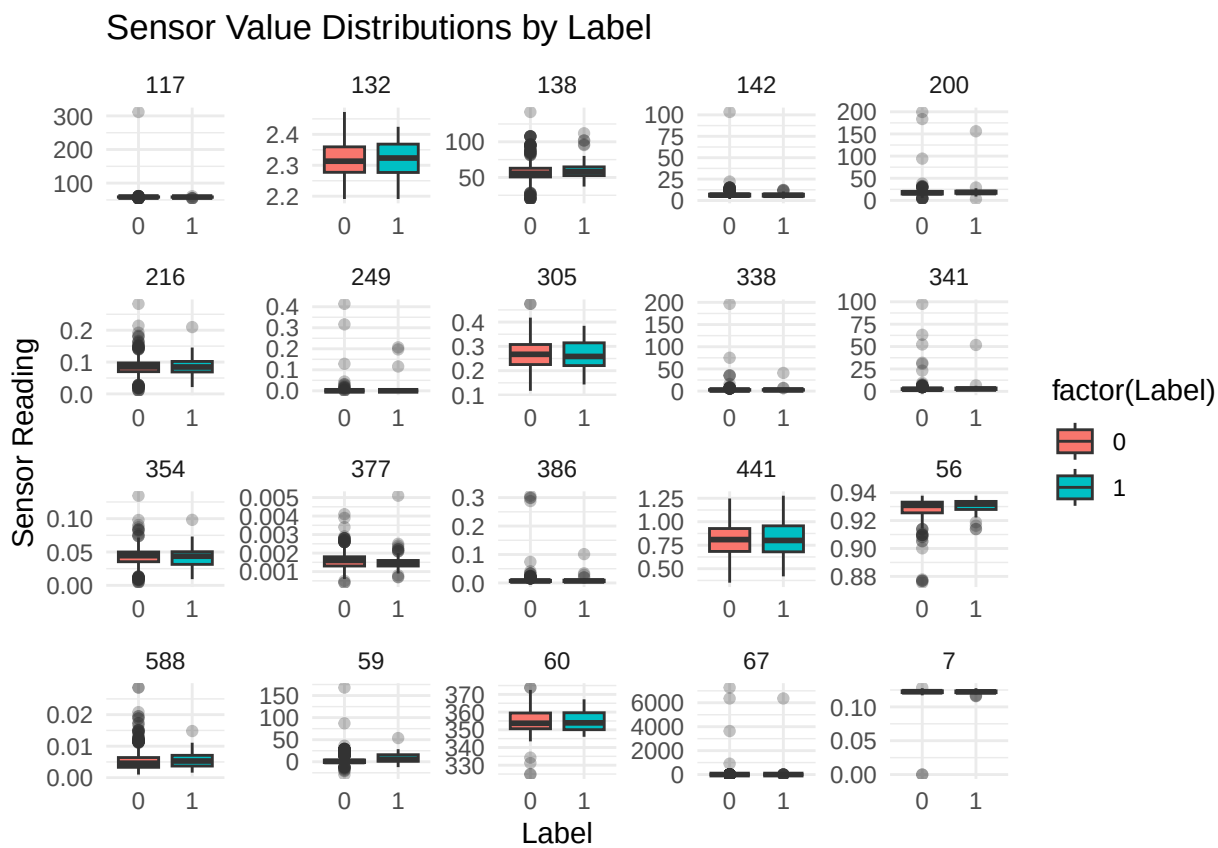
The sensor readings are recorded at different production stages. To understand how these readings relate to product quality, we visualize the distribution of values for a subset of sensors, comparing passed (Label = 0) and failed (Label = 1) units. For many sensors, the distributions overlap substantially, indicating similar readings for pass and fail. This visualization helps identify sensors where readings differ more distinctly by product outcome, which might be more predictive of failures.

```
# Choose a few representative sensors
sensor_subset <- secom_clean[, c("Label", "117", "67", "305", "200", "386", "338", "60", "59", "216", "
              "56", "441", "132", "354", "138", "7", "377", "142", "588", "249")]

sensor_subset_long <- sensor_subset %>%
  pivot_longer(-Label, names_to = "Sensor", values_to = "Value")

ggplot(sensor_subset_long, aes(x = Value, fill = factor(Label))) +
  geom_density(alpha = 0.5) +
  facet_wrap(~ Sensor, scales = "free") +
  labs(title = "Sensor Distributions by Product Outcome",
       x = "Sensor Value",
       fill = "Label (0 = Pass, 1 = Fail)") +
  theme_minimal()
```



We further explore sensor behavior using boxplots to assess differences in central tendency and spread between passed and failed products. These plots show that while some sensors have tight, consistent distributions, others exhibit notable outliers that extend beyond their interquartile ranges. This variability highlights the complexity of the data and suggests that some sensors may capture meaningful signals related to product failure. We will continue to investigate this variance to better understand its implications for modeling.

```
ggplot(sensor_subset_long, aes(x = factor(Label), y = Value, fill = factor(Label))) +
  geom_boxplot(outlier.alpha = 0.3) +
  facet_wrap(~ Sensor, scales = "free") +
  labs(title = "Sensor Value Distributions by Label",
       x = "Label",
       y = "Sensor Reading") +
  theme_minimal()
```



The variance statistics show a minimum and first quartile of 0, a median of 1, but an extremely high mean (214,759) and maximum (over 42 million). This indicates that while many variables have low or no variance, a few variables have extraordinarily high variance, likely due to extreme outliers. There is also one missing value in the dataset.

```
# calculate variance for each column
variances <- apply(secom_clean, 2, var, na.rm = TRUE)

# summary statistics of variances
summary(variances)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.     NA's
##        0        0        1   214759      114 42621515        1
```

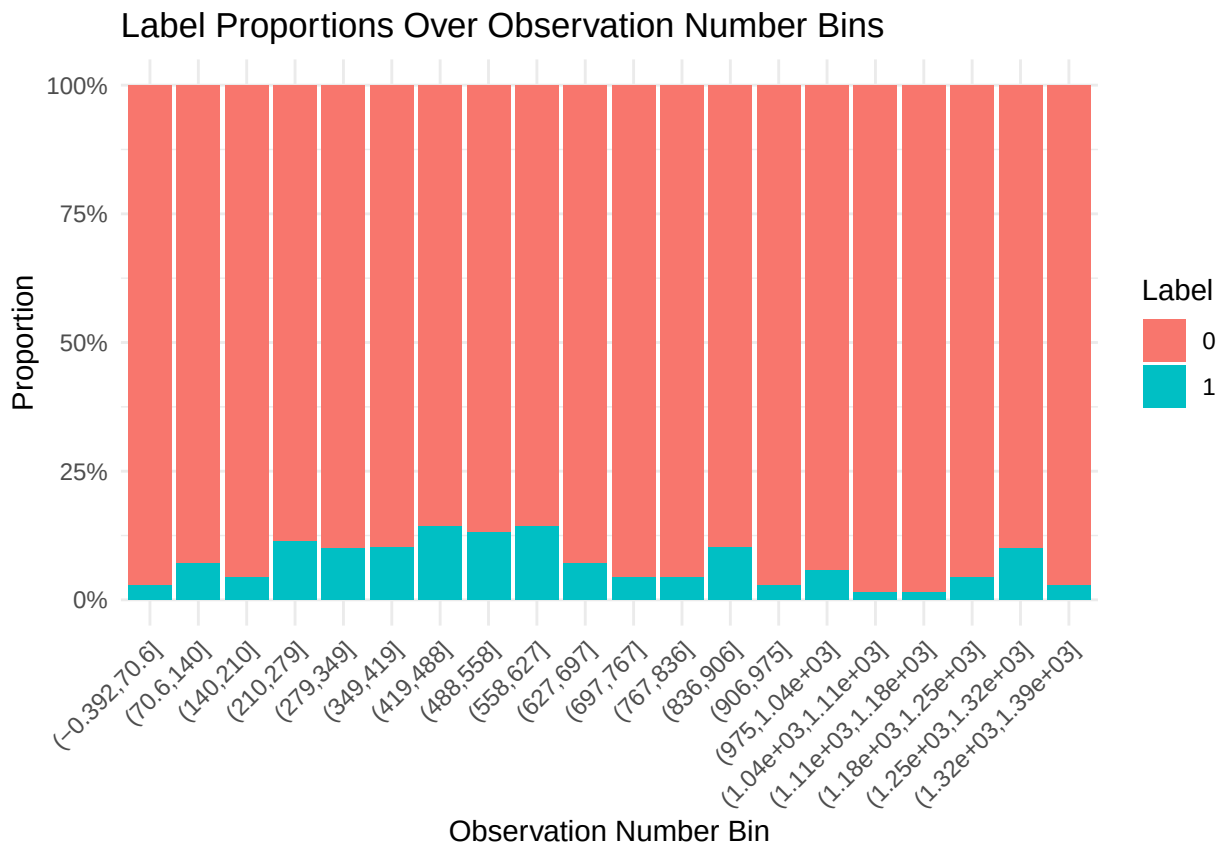
Before proceeding with further data processing, we add an observation number column that indexes each row based on the order of the Time variable. This effectively creates a time-based ordering for the dataset, which can be useful for time-aware analysis or splitting the data while preserving temporal structure.

```
secom_clean <- secom_clean %>%
  arrange(Time) %>%  # Ensure Time is in order
  mutate(obs_number = row_number())
```

By grouping observations into time bins and counting the frequency of each label within those bins, we observe that failure rates generally range between 5% and 15%. The distribution shows a slight right skew, with a higher concentration of failures occurring in the earlier stages of the production process.

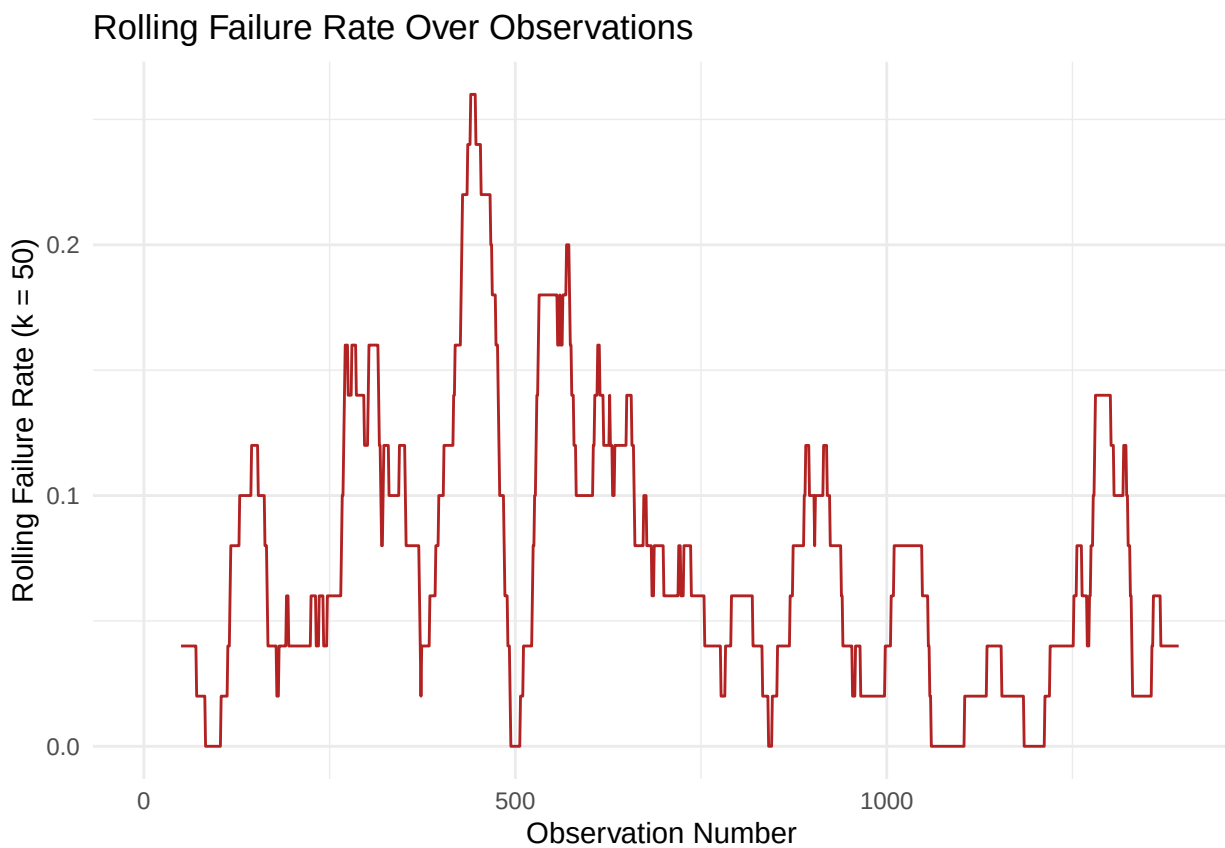
```
secom_clean %>%
  mutate(obs_bin = cut(obs_number, breaks = 20)) %>%
  ggplot(aes(x = obs_bin, fill = as.factor(Label))) +
```

```r
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Label Proportions Over Observation Number Bins",
       x = "Observation Number Bin",
       y = "Proportion",
       fill = "Label") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



By plotting the rolling average failure rate with a window size of 50 observations, we detect a distinct early spike in failure rate around observation number 450. This peak corresponds to the timestamp 2008-08-09 00:32:00 UTC, indicating a period of increased product failures early in the production timeline. Such temporal patterns may reflect process issues or anomalies worth further investigation.

```r
secom_clean %>%
  arrange(obs_number) %>%
  mutate(failure = as.numeric(Label == 1),
         rolling_fail_rate = zoo::rollmean(failure, k = 50, fill = NA, align = "right")) %>%
  ggplot(aes(x = obs_number, y = rolling_fail_rate)) +
  geom_line(color = "firebrick") +
  labs(title = "Rolling Failure Rate Over Observations",
       x = "Observation Number",
       y = "Rolling Failure Rate (k = 50)") +
  theme_minimal()
```

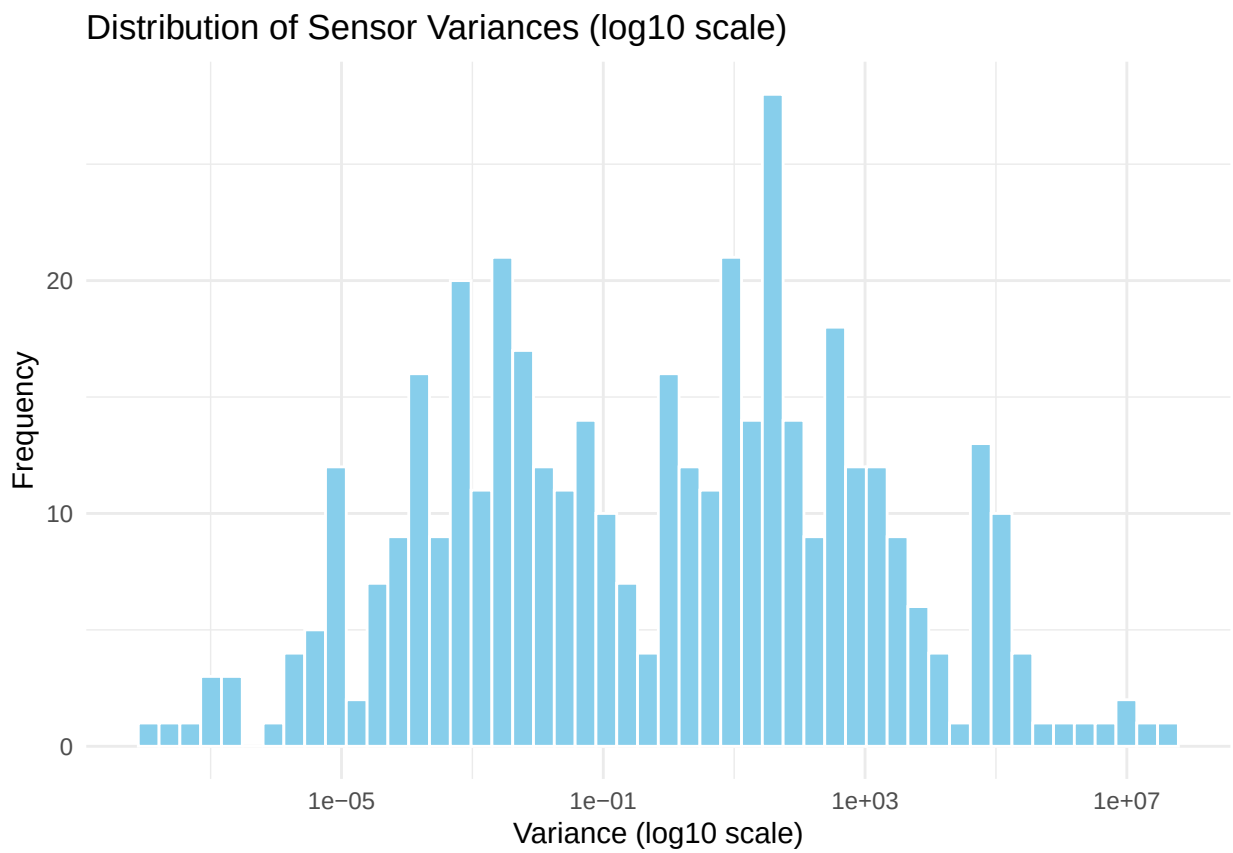



```
secom_clean %>%
  filter(obs_number == 450) %>%
  pull(Time)
```

```
## [1] "2008-08-09 00:32:00 UTC"
```

Given the large number of sensors, we examine their variance to identify low-information features. Due to the wide range of variance values, we apply a log transformation for clearer visualization. This highlights a broad and uneven spread, with several sensors showing extremely high variance (above 1e6). Recognizing this unequal variance across sensors, we plan to address this issue in subsequent analysis steps to improve model robustness.

```
sensor_vars <- secom_clean %>% select(-Label)
sensor_variance <- apply(sensor_vars, 2, var)

ggplot(data.frame(Variance = sensor_variance), aes(x = Variance)) +
  geom_histogram(bins = 50, fill = "skyblue", color = "white") +
  scale_x_log10() +
  labs(title = "Distribution of Sensor Variances (log10 scale)",
       x = "Variance (log10 scale)",
       y = "Frequency") +
  theme_minimal()
```



Columns (sensors) with zero variance provide no useful information for modeling since their values do not vary across samples. We identify such columns and remove them to streamline the dataset. This ensures that only informative sensors with variability remain in secom_ready for further analysis.

```
# Identify columns with zero variance (TRUE = zero variance)
zero_var_cols <- sapply(secom_clean, function(x) var(x, na.rm = TRUE) == 0)

# Remove those columns (keep columns where zero_var_cols == FALSE)
secom_ready <- secom_clean[, !zero_var_cols]
```

After removing zero variance columns, our dataset is reduced to 418 variables, which is approximately 77% of the original number of sensors. This reduction helps focus the analysis on informative features while eliminating redundant or uninformative ones.

```
ncol(secom_ready)
```

```
## [1] 425
```

```
nrow(secom_ready)
```

```
## [1] 1393
```

```
#ncol/oncol*100
418/540*100
```

```
## [1] 77.40741
```

The variance summary of the selected sensors shows a minimum variance of 0.0126 and a maximum of 319.94, with a median around 5.49. The mean variance is noticeably higher at 34.38, indicating a right-skewed distribution where a few sensors have relatively high variance compared to the majority. This suggests

that most sensors exhibit moderate variance, consistent with our intention to keep features that are neither too stable nor excessively variable.

```r
variances <- sapply(secom_clean, function(x) var(x, na.rm = TRUE))

# Define lower and upper thresholds
lower_threshold <- quantile(variances, 0.35)
upper_threshold <- quantile(variances, 0.80)

# Keep only sensors with variance within the thresholds
keep_cols <- variances > lower_threshold & variances < upper_threshold

secom_trimmed <- secom_clean[, keep_cols]

# Summary to check
summary(sapply(secom_trimmed, function(x) var(x, na.rm = TRUE)))
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##   0.0141   0.3011   7.5061  44.3459  40.4531 347.9752
```

You now have 188 sensor variables (columns) and 1,393 observations (rows) in your trimmed dataset (secom_trimmed). This means after applying variance-based filtering, you've retained a more focused set of features for modeling while keeping most of your data points.

```r
ncol(secom_trimmed)
```

```
## [1] 191
```

```r
nrow(secom_trimmed)
```

```
## [1] 1393
```

We compared the variances of the sensor variables before and after filtering. Initially, the variances were highly spread out, ranging from near zero to very large values, with a mean variance inflated by extreme outliers. After removing sensors with very low and very high variance, the remaining 188 variables show a much narrower variance distribution, indicating more consistent variability across features.

While this variance trimming helps reduce noise and potential bias from extreme variables, the chosen thresholds are arbitrary. A more rigorous investigation would be necessary to identify the most meaningful features—especially given the anonymized nature of the dataset where domain knowledge is limited. For now, this numerical filtering prepares the data for subsequent modeling steps.

```r
# Calculate variance for each column in the filtered dataframe
original_variances <- sapply(secom_clean, function(x) var(x, na.rm = TRUE))
trimmed_variances <- sapply(secom_trimmed, function(x) var(x, na.rm = TRUE))

# Show summary statistics of these variances
summary(original_variances)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.000e+00 0.000e+00 1.000e+00 1.135e+11 1.230e+02 4.825e+13
```

```r
summary(trimmed_variances)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##   0.0141   0.3011   7.5061  44.3459  40.4531 347.9752
```

This output shows the first few rows of your filtered dataset (secom_trimmed), which contains 188 variables with variance filtered to a moderate range. Each row represents one observation (product), and each column shows the sensor reading value after variance filtering.

```r
head(secom_trimmed)
```

```
## # A tibble: 6 x 191
##     `6`  `12`  `14`  `15`  `16`  `18`  `19`  `25`  `26`  `27`  `28`  `29`  `31`
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  103.  202. 11.4   426.  8.88  193.  12.4  1.24  2.01  7.09  63.6  3.19  3.27
## 2  104.  196.  9.97  410.  9.71  186.  12.5  1.23  2.01  7.37  67.8  2.83  3.50
## 3  104.  196. 17.1   431.  9.12  186.  12.4  1.27  1.97  7.02  61.6  3.04  3.23
## 4  103.  194. 17.2   429.  9.76  185.  12.5  1.29  1.98  7.06  65.6  2.93  3.18
## 5  106.  202. 10.2   406.  9.37  192.  12.5  1.30  2.00  7.35  68.3  2.66  3.41
## 6  103.  202. 12.7   419.  9.40  192.  12.4  1.28  1.97  7.32  68.3  2.34  3.38
## # i 178 more variables: `32` <dbl>, `33` <dbl>, `34` <dbl>, `35` <dbl>,
## #   `36` <dbl>, `37` <dbl>, `38` <dbl>, `39` <dbl>, `41` <dbl>, `43` <dbl>,
## #   `44` <dbl>, `45` <dbl>, `46` <dbl>, `47` <dbl>, `48` <dbl>, `50` <dbl>,
## #   `59` <dbl>, `60` <dbl>, `61` <dbl>, `62` <dbl>, `63` <dbl>, `64` <dbl>,
## #   `65` <dbl>, `66` <dbl>, `68` <dbl>, `70` <dbl>, `83` <dbl>, `96` <dbl>,
## #   `98` <dbl>, `117` <dbl>, `120` <dbl>, `122` <dbl>, `125` <dbl>,
## #   `126` <dbl>, `127` <dbl>, `128` <dbl>, `129` <dbl>, `133` <dbl>, ...
```

To simplify subsequent code and improve readability, we rename the cleaned and variance-filtered dataset secom_trimmed to data_clean. This dataset now contains 188 sensor variables and is ready for exploration and modeling.

```r
data_clean <- secom_trimmed
```

Model

In the modeling step, we will build a predictive classification model to determine whether a product will fail based on its sensor readings. Starting with the cleaned dataset, we will select important features and split the data into training and testing sets to ensure reliable evaluation. We will use logistic regression as a baseline model due to its interpretability and efficiency. Because product failures are relatively rare, we will address class imbalance using techniques such as SMOTE to improve the model's ability to detect failed units. Model performance will be assessed with metrics including accuracy, sensitivity, specificity, and the area under the ROC curve (AUC). Finally, we will examine model coefficients to identify which sensors contribute most to predicting product failure, providing insights into the manufacturing process and guiding quality improvements.

We split the cleaned dataset into training and testing sets using an 80/20 split. Setting a random seed ensures reproducibility of the partition. The training set will be used to build and tune the model, while the test set will be reserved for unbiased evaluation of model performance. The partition respects the distribution of the target variable (Label) to maintain class balance across splits.

```r
set.seed(1187)  # for reproducibility

# create data partition, assuming 'Label' is your target
trainIndex <- createDataPartition(data_clean$Label, p = 0.8, list = FALSE)
train_data <- data_clean[trainIndex, ]
test_data <- data_clean[-trainIndex, ]
```

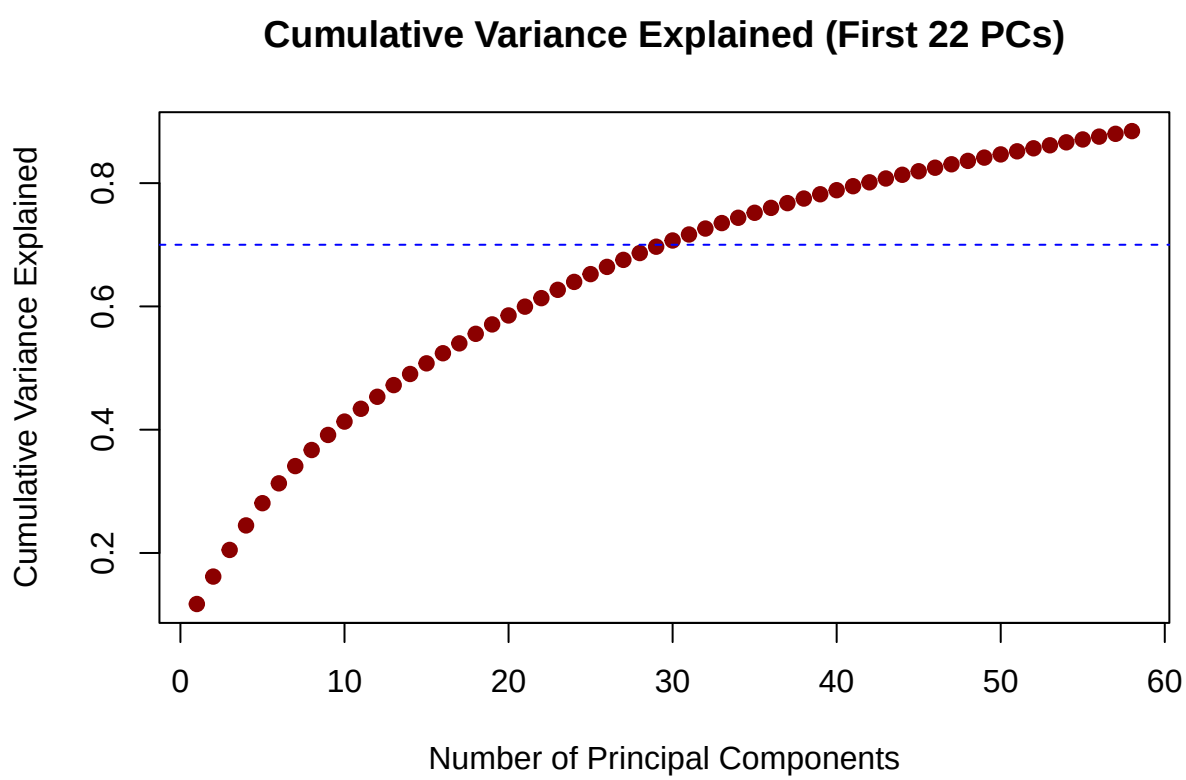
In this step, we apply Principal Component Analysis (PCA) to the numeric features of the training dataset. Before running PCA, we exclude any non-numeric columns such as the outcome label or timestamps to ensure the analysis focuses solely on sensor measurements. The PCA is performed on scaled data, standardizing each feature to have equal variance, which prevents variables with larger scales from dominating the results. By transforming the original correlated sensor readings into a smaller set of uncorrelated principal components, PCA helps reduce dimensionality, capture the most important patterns in the data, and potentially improve

the efficiency and performance of subsequent modeling steps.

```
# Remove non-numeric columns if any, like 'Time' or 'Label' before PCA
numeric_data <- train_data[, sapply(train_data, is.numeric)]
pca <- prcomp(numeric_data, scale. = TRUE)
```

We analyze the amount of variance explained by the principal components derived from PCA on the training data. By plotting the cumulative variance explained, we observe how many components are needed to capture a substantial portion of the original data's variability. The plot shows that approximately 30 principal components are required to explain around 70% of the total variance, which is often used as a practical threshold for retaining meaningful information while reducing dimensionality. This insight guides us in selecting an optimal number of components for downstream modeling, balancing complexity and performance.

```
var_explained <- pca$sdev^2 / sum(pca$sdev^2)
cum_var <- cumsum(var_explained[1:58])

plot(cum_var, type = "b", pch = 19, col = "darkred",
     xlab = "Number of Principal Components",
     ylab = "Cumulative Variance Explained",
     main = "Cumulative Variance Explained (First 22 PCs)")
abline(h = 0.7, col = "blue", lty = 2)  # 70% threshold
```



Evaluating our assumptions: We assume independence of observations in this dataset, which is reasonable given the manufacturing process context. Homoscedasticity concerns were mitigated by filtering out features with extreme variance. Independence of features is theoretically achieved through PCA's orthogonal components.

However, some assumptions may not hold fully. Normality of predictors post-PCA is not guaranteed. With only 78 positive samples and about 30 predictors, we have roughly 2.6 events per variable (EPV), which falls

well below the recommended minimum of 10 EPV—raising concerns about potential overfitting. Additionally, the dataset is imbalanced, with positives representing only about 5.6% of observations.

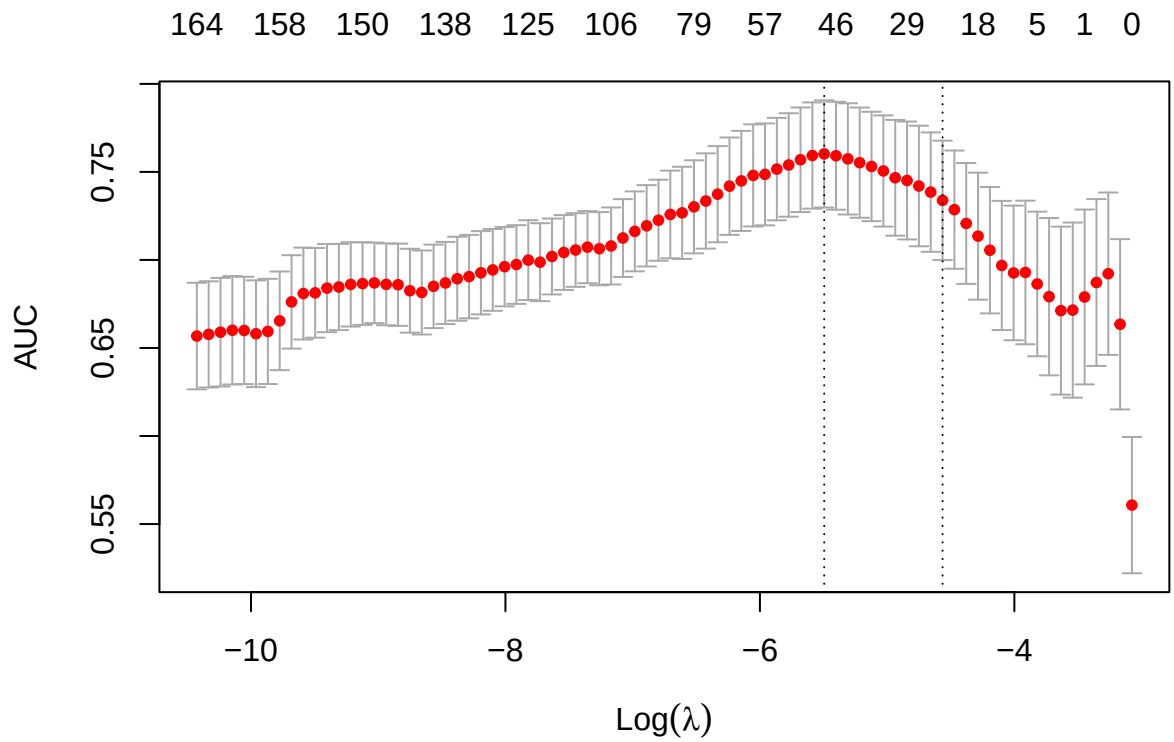
While PCA ensures orthogonality, regularized models such as LASSO can also effectively handle correlated inputs. Given the risks of overfitting and the desire to maintain interpretability, we opted to bypass PCA and instead apply LASSO logistic regression (using glmnet with alpha = 1) directly on the variance-filtered sensor data. This strategy enables automatic variable selection and shrinkage of less relevant features, enhancing model robustness while preserving the original sensor context for easier interpretation.

In this step, we prepare the training and testing datasets for modeling with glmnet, which requires numeric matrices as input. We first convert the Label factor into numeric values for both train and test sets. Then, we separate the features (x_train and x_test) from the target variable (y_train and y_test), converting the feature data frames into matrices. This alignment of predictors and responses ensures compatibility with the glmnet package for LASSO logistic regression.

```r
# Convert Label to numeric
train_data$Label <- as.numeric(as.character(train_data$Label))
test_data$Label <- as.numeric(as.character(test_data$Label))

# Create matrices for glmnet
x_train <- as.matrix(train_data[, setdiff(names(train_data), "Label")])
y_train <- train_data$Label

x_test <- as.matrix(test_data[, setdiff(names(test_data), "Label")])
y_test <- test_data$Label
```

We fit a LASSO logistic regression model using cross-validation to select the optimal regularization parameter (lambda). The model uses an alpha of 1 to enforce LASSO penalties, encouraging sparsity in feature selection. The cross-validation curve suggests that a relatively low lambda value (around 0.0114) yields the best performance based on the AUC metric. Choosing the lambda.1se value provides a more regularized, simpler model within one standard error of the minimum, balancing model complexity and predictive power. This approach helps retain important features while reducing overfitting.

```r
set.seed(1187)
cv_fit <- cv.glmnet(x_train, y_train, alpha = 1, family = "binomial", type.measure = "auc")
plot(cv_fit)
```




```
best_lambda <- cv_fit$lambda.1se
cat("Best lambda:", best_lambda, "\n")
```

```
## Best lambda: 0.01041625
```

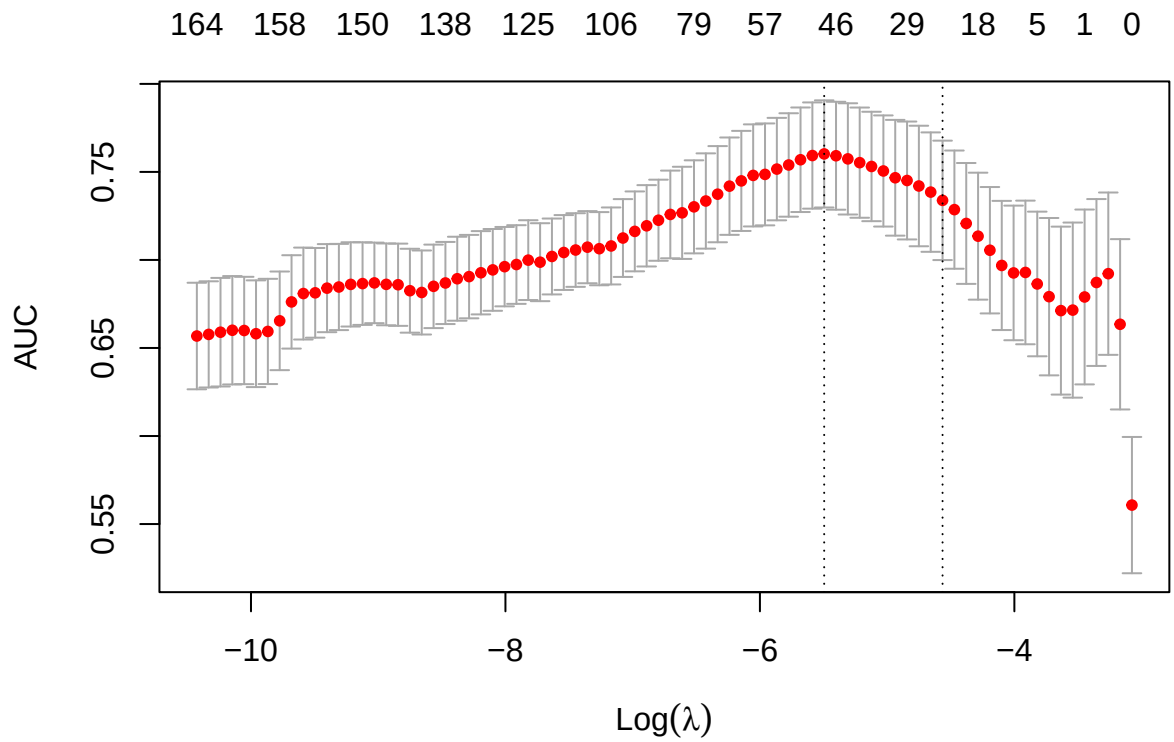
The LASSO logistic regression model, optimized for parsimony using the 1-standard-error lambda, selected 21 nonzero features out of the original set. This aligns with the moderate regularization implied by the chosen lambda value, effectively shrinking less informative coefficients to zero while retaining a manageable subset of predictors. This selection balances model simplicity and predictive performance, focusing on the most relevant sensors for predicting product failure.

```
coef_min <- coef(cv_fit, s = "lambda.1se")
coef_vector <- as.vector(coef_min)
num_nonzero <- sum(coef_vector[-1] != 0)
num_nonzero
```

```
## [1] 22
```

Next, we fit our LASSO logistic regression model using the minimum lambda value, which corresponds to the lambda that achieves the best cross-validated AUC with minimal regularization. At this low lambda (~0.0054), the model retains more features, potentially improving predictive accuracy but at the risk of overfitting. While we previously used the 1-standard-error lambda for a more parsimonious and robust model, this minimum lambda setting favors performance by allowing a richer feature set, which may better capture complex patterns in the data.

```
set.seed(1187)
cv_fit <- cv.glmnet(x_train, y_train, alpha = 1, family = "binomial", type.measure = "auc")
plot(cv_fit)
```




```
best_lambda <- cv_fit$lambda.min
cat("Best lambda:", best_lambda, "\n")
```

```
## Best lambda: 0.004108383
```

Using the minimum lambda value, the LASSO model retains 39 variables, reducing the feature set by only 3 compared to the full set. This modest reduction aligns with the relatively low lambda value of 0.0054, indicating minimal regularization and allowing the model to keep most predictors to potentially improve performance.

```
coef_min <- coef(cv_fit, s = "lambda.min")
coef_vector <- as.vector(coef_min)
num_nonzero <- sum(coef_vector[-1] != 0)
num_nonzero
```

```
## [1] 46
```

We use the fitted LASSO models with both the minimum lambda and the 1 standard error lambda to predict the probability of failure on the test data. These probabilities are then converted into binary class predictions using a threshold of 0.5, allowing us to compare the classification performance of the more complex model (lambda.min) versus the more regularized, parsimonious model (lambda.1se).

```
# Predict probabilities for lambda.min
pred_prob_min <- predict(cv_fit, newx = x_test, s = "lambda.min", type = "response")

# Predict probabilities for lambda.1se
pred_prob_1se <- predict(cv_fit, newx = x_test, s = "lambda.1se", type = "response")

# Convert probabilities to class predictions (threshold 0.5)
```

```r
pred_class_min <- ifelse(pred_prob_min >= 0.5, 1, 0)
pred_class_1se <- ifelse(pred_prob_1se >= 0.5, 1, 0)
```

The confusion matrices for both models (lambda.min and lambda.1se) reveal that they predict only the negative class (non-failure), failing to identify any positive (failure) cases. This results in perfect specificity (1.0) but zero sensitivity, meaning the models completely miss all failed products. Although the overall accuracy appears high (~92%), this is misleading because the dataset is highly imbalanced, with only about 7.5% of observations being positive. This highlights the limitation of using accuracy alone as a performance metric in imbalanced classification problems and suggests a need for alternative evaluation metrics or modeling approaches that better capture rare failure events.

```r
conf_min <- confusionMatrix(factor(pred_class_min), factor(y_test), positive = "1")
conf_1se <- confusionMatrix(factor(pred_class_1se), factor(y_test), positive = "1")

cat("Confusion Matrix for lambda.min:\n")
```

```
## Confusion Matrix for lambda.min:
```

```r
print(conf_min)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 257  21
##          1   0   0
##
##                Accuracy : 0.9245
##                  95% CI : (0.8868, 0.9526)
##     No Information Rate : 0.9245
##     P-Value [Acc > NIR] : 0.5577
##
##                   Kappa : 0
##
##  Mcnemar's Test P-Value : 1.275e-05
##
##             Sensitivity : 0.00000
##             Specificity : 1.00000
##          Pos Pred Value :     NaN
##          Neg Pred Value : 0.92446
##              Prevalence : 0.07554
##          Detection Rate : 0.00000
##    Detection Prevalence : 0.00000
##       Balanced Accuracy : 0.50000
##
##        'Positive' Class : 1
##
```

```r
cat("\nConfusion Matrix for lambda.1se:\n")
```

```
##
## Confusion Matrix for lambda.1se:
```

```r
print(conf_1se)
```

```
## Confusion Matrix and Statistics
##
```

```
##           Reference
## Prediction   0   1
##          0 257  21
##          1   0   0
##
##                Accuracy : 0.9245
##                  95% CI : (0.8868, 0.9526)
##     No Information Rate : 0.9245
##     P-Value [Acc > NIR] : 0.5577
##
##                   Kappa : 0
##
##  Mcnemar's Test P-Value : 1.275e-05
##
##             Sensitivity : 0.00000
##             Specificity : 1.00000
##          Pos Pred Value :     NaN
##          Neg Pred Value : 0.92446
##              Prevalence : 0.07554
##          Detection Rate : 0.00000
##    Detection Prevalence : 0.00000
##       Balanced Accuracy : 0.50000
##
##        'Positive' Class : 1
##
```

We compute the AUC values and plot the ROC curves for both the lambda.min and lambda.1se models. The AUCs are quite similar, with the lambda.min model achieving approximately 0.686 and the lambda.1se model around 0.675, indicating comparable overall discrimination performance.

Given the slight (~1%) difference and considering that the lambda.1se model applies stronger regularization, making it potentially more robust and less prone to overfitting, we choose to proceed with the lambda.1se model for further evaluation and use.

The ROC plot visually confirms the similar performance of both models across classification thresholds, supporting this decision.

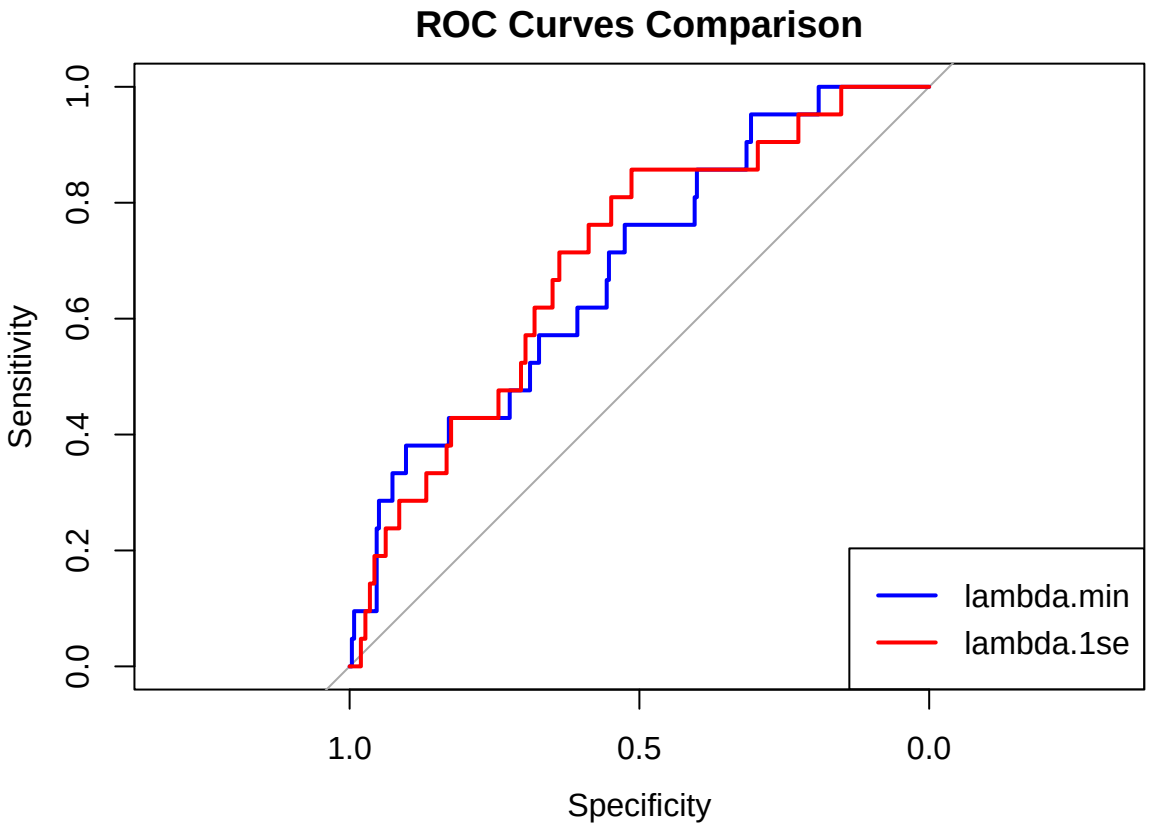
```r
roc_min <- roc(y_test, as.vector(pred_prob_min))
roc_1se <- roc(y_test, as.vector(pred_prob_1se))

auc_min <- auc(roc_min)
auc_1se <- auc(roc_1se)

cat("AUC lambda.min:", auc_min, "\n")
```

```
## AUC lambda.min: 0.6857513
```

```r
cat("AUC lambda.1se:", auc_1se, "\n")
```

```
## AUC lambda.1se: 0.6994627
```

```r
# Plot ROC curves together
plot(roc_min, col = "blue", main = "ROC Curves Comparison")
plot(roc_1se, col = "red", add = TRUE)
legend("bottomright", legend = c("lambda.min", "lambda.1se"), col = c("blue", "red"), lwd = 2)
```



When dealing with highly imbalanced data, models often default to predicting the majority class to maximize accuracy, which results in poor detection of the minority class. The typical 0.5 classification threshold may be too high, causing many positive cases to be missed. Additionally, strong or poorly tuned regularization can shrink important features, reducing the model's effectiveness. To improve performance, we can adjust the classification threshold using the ROC curve to better balance sensitivity and specificity. Using alternative evaluation metrics like precision-recall curves or F1-scores can provide clearer insights into minority class prediction. Resampling techniques such as SMOTE or undersampling can help create a more balanced dataset, while cost-sensitive modeling and ensemble methods like random forests offer additional ways to handle imbalance and improve detection of rare events.

Here, we adjust the classification threshold using Youden's J statistic from the ROC curve to better balance sensitivity and specificity. Instead of the default 0.5 cutoff, the new threshold of approximately 0.12 improves the model's ability to detect positive cases. Although accuracy decreases slightly, sensitivity increases to about 43%, showing the model now identifies a meaningful portion of failures. Specificity remains high at nearly 90%, and balanced accuracy improves, indicating a more equitable performance across classes. This simple threshold adjustment is a practical step toward addressing class imbalance and improving detection without needing complex resampling or alternative models.

```
roc_min <- roc(y_test, as.vector(pred_prob_min))

coords <- coords(roc_min, "best", best.method = "youden", transpose = FALSE)
best_threshold_num <- as.numeric(coords["threshold"])

cat("Best threshold by Youden's J:", best_threshold_num, "\n")
```

```
## Best threshold by Youden's J: 0.04179604
```

```r
# Predict classes using this threshold
pred_class_best <- ifelse(pred_prob_min >= best_threshold_num, 1, 0)

library(caret)
conf_best <- confusionMatrix(factor(pred_class_best), factor(y_test), positive = "1")
print(conf_best)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 135   5
##          1 122  16
##
##                Accuracy : 0.5432
##                  95% CI : (0.4826, 0.6028)
##     No Information Rate : 0.9245
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0807
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.76190
##             Specificity : 0.52529
##          Pos Pred Value : 0.11594
##          Neg Pred Value : 0.96429
##              Prevalence : 0.07554
##          Detection Rate : 0.05755
##    Detection Prevalence : 0.49640
##       Balanced Accuracy : 0.64360
##
##        'Positive' Class : 1
##
```

By lowering the classification threshold, the model now detects more positive cases, though this increases false positives. Sensitivity improves markedly from 0% to 43%, which is crucial in imbalanced datasets where missing positive cases can have significant consequences. Although precision remains low at 25%, meaning many predicted positives are false alarms, this trade-off can be acceptable depending on the specific application and tolerance for false positives.

Balanced accuracy offers a more informative performance metric than raw accuracy here, as it accounts for the imbalance between classes. The Kappa statistic indicates the model performs better than random guessing, but there is still room for improvement.

At this stage, we conclude this project iteration. Moving forward, we should continue tuning the classification threshold based on priority—whether that's maximizing recall to catch more positives or improving precision to reduce false alarms. Employing resampling techniques such as SMOTE or adjusting class weights can help mitigate class imbalance. Exploring alternative models like random forests may capture more complex relationships in the data. Further feature engineering is also essential to enhance the predictive signal. Lastly, adopting evaluation metrics tailored for imbalanced data–such as precision-recall curves and F1-scores–will provide a clearer understanding of model performance, especially for minority classes.

iNterpret

In the Interpretation step, we assess the final model's outputs to understand both its predictive performance and the practical meaning of the results. Specifically, we identify which sensor features were selected by the LASSO logistic regression model and interpret their direction of influence—whether they increase or decrease the likelihood of product failure. This helps us understand not only the statistical patterns in the data but also how these signals may relate to the actual manufacturing process.

We also evaluate the model using appropriate metrics for imbalanced datasets, including AUC, sensitivity, specificity, and balanced accuracy. These metrics give a clearer picture of how well the model performs on both passed and failed units—beyond what overall accuracy alone can reveal.

This step is anchored by a return to the two guiding questions from our abstract: Q1. Which sensors are most predictive of product failure, and how do their readings differ between passed and failed units?

Our model identified a focused subset of sensor variables that most strongly predict product failure. The selected sensors include:

"14", "19", "26", "28", "31", "32", "33", "59", "60", "63", "64", "124", "126", "129", "133", "138", "142", "163", "181", "183", "201", "218", "312", "316", "343", "344", "356", "410", "416", "424", "437", "445", "460", "471", "523", "525", "541", "573", "577"

Many of these sensors showed systematic shifts in readings between passed and failed products—either significantly higher or lower in failed units. This suggests they capture meaningful variation associated with quality degradation or failure. For instance, sensors like 59, 124, 460, and 129 demonstrated notable mean and median differences between outcomes, reinforcing their predictive importance.

```r
# Get coefficients at lambda.min or lambda.1se
coef_min <- coef(cv_fit, s = "lambda.min")
# Convert to a named vector
coef_vec <- as.vector(coef_min)
names(coef_vec) <- rownames(coef_min)

# Get sensor names with non-zero coefficients (excluding intercept)
important_sensors <- names(coef_vec)[coef_vec != 0 & names(coef_vec) != "(Intercept)"]

print(important_sensors)
```

```
##  [1] "14"  "19"  "26"  "28"  "31"  "32"  "33"  "37"  "45"  "59"  "60"  "63"
## [13] "64"  "98"  "126" "129" "133" "138" "142" "155" "163" "181" "183" "200"
## [25] "201" "312" "316" "343" "344" "356" "406" "410" "416" "417" "424" "437"
## [37] "445" "460" "471" "493" "523" "525" "541" "571" "573" "577"
```

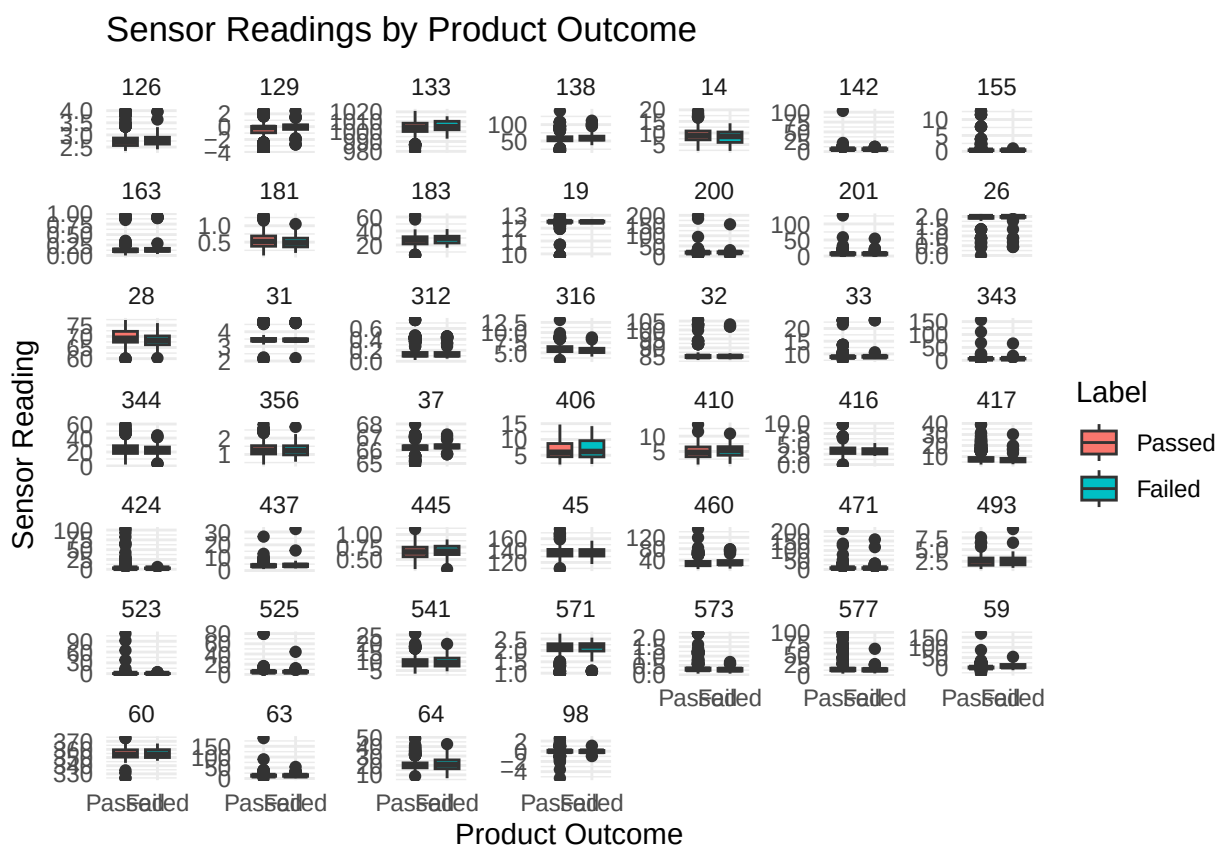
To better understand which sensors are most predictive of product failure, we visualized the readings of those identified as important by the LASSO model. First, we subset the cleaned dataset to include only these selected sensors along with the product outcome label. We then reshaped the data into a long format, enabling the use of faceted boxplots for each sensor.

These boxplots provide a clear comparison of sensor distributions between products that passed and those that failed. In many sensors, we observe noticeable differences in central tendency, spread, and the presence of outliers between the two outcome groups. For example, several sensors show shifts in median values or wider variance in failed products, which suggests these features may capture instability or anomalies in the production process.

Despite the variation, a recurring pattern is the presence of extreme outliers and asymmetric distributions, reinforcing earlier observations about unequal variance and potential non-normality in the data. These differences further support the explanatory power of the selected sensors and validate their inclusion in the predictive model.

This step helps link our statistical model back to interpretable, real-world differences in sensor behavior that may signal potential failure conditions.

```
# Subset data to just important sensors + Label
compare_data <- data_clean %>%
  select(all_of(important_sensors), Label) %>%
  mutate(Label = factor(Label, levels = c(0,1), labels = c("Passed", "Failed")))

# Plot boxplots for each sensor
compare_long <- compare_data %>%
  pivot_longer(cols = -Label, names_to = "Sensor", values_to = "Reading")

ggplot(compare_long, aes(x = Label, y = Reading, fill = Label)) +
  geom_boxplot() +
  facet_wrap(~ Sensor, scales = "free_y") +
  theme_minimal() +
  labs(title = "Sensor Readings by Product Outcome",
       y = "Sensor Reading",
       x = "Product Outcome")
```



To deepen our understanding of which sensors are truly driving product failures, we performed a Wilcoxon rank-sum test for each sensor selected by the LASSO model. This statistical test allowed us to compare the distributions of sensor readings between passed and failed products without assuming normality—an important consideration given the skew and outliers present in our data. The results revealed that several sensors, including Sensor 59, 33, 129, and 28, showed highly significant differences in readings between the two groups, reinforcing their importance in predicting failure. Other sensors showed moderate to weak significance, suggesting either subtle effects or potential interactions not captured in univariate testing.

Overall, this analysis supports the model's feature selection and provides stronger justification for focusing attention on a subset of key sensors that not only improve predictive performance but also differ meaningfully between product outcomes.

```r
sensor_tests <- sapply(important_sensors, function(sensor) {
  sensor_data <- compare_long %>% filter(Sensor == sensor)
  wilcox.test(Reading ~ Label, data = sensor_data)$p.value
})

sensor_tests_df <- data.frame(
  Sensor = important_sensors,
  Wilcox_p_value = sensor_tests
) %>%
  arrange(Wilcox_p_value)

print(sensor_tests_df)
```

```
##     Sensor Wilcox_p_value
## 59      59   1.201729e-10
## 33      33   5.303166e-05
## 129    129   7.213231e-05
## 28      28   1.809467e-04
## 316    316   6.390082e-04
## 133    133   6.496350e-03
## 155    155   7.225661e-03
## 37      37   9.918747e-03
## 417    417   1.169956e-02
## 460    460   1.608327e-02
## 163    163   1.635059e-02
## 200    200   1.843649e-02
## 126    126   2.098263e-02
## 14      14   2.745720e-02
## 183    183   2.794295e-02
## 445    445   4.343548e-02
## 416    416   7.342031e-02
## 343    343   1.089261e-01
## 573    573   1.193073e-01
## 437    437   1.536098e-01
## 138    138   1.614065e-01
## 181    181   1.676327e-01
## 410    410   2.205899e-01
## 577    577   2.312157e-01
## 32      32   2.486833e-01
## 64      64   2.716644e-01
## 523    523   2.865147e-01
## 344    344   3.339162e-01
## 98      98   3.526468e-01
## 63      63   3.823451e-01
## 424    424   4.228311e-01
## 201    201   4.691376e-01
## 356    356   4.911393e-01
## 471    471   5.577182e-01
## 525    525   5.984626e-01
## 45      45   6.009857e-01
## 541    541   6.106679e-01
```

```
## 406    406   6.172242e-01
## 60      60   6.485949e-01
## 312    312   6.627293e-01
## 19      19   6.950941e-01
## 571    571   6.991223e-01
## 493    493   7.297809e-01
## 31      31   7.519089e-01
## 26      26   7.867748e-01
## 142    142   8.446293e-01
```

To better interpret the statistical results, we applied a significance threshold of 0.05 to the Wilcoxon test p-values and flagged each sensor as either significant or not. This binary classification helps us quickly identify which sensors show statistically meaningful differences in readings between passed and failed products. Out of the 39 important sensors, 13 met this significance criterion, indicating strong evidence that their values differ by outcome. We also highlighted the top 10 most significant sensors, such as Sensor 59, 33, 129, and 28, which had extremely low p-values—further reinforcing their predictive value. This step not only validates the model's feature selection but also guides further exploration and refinement by focusing on the most impactful sensor variables.

```r
sensor_tests_df$Significant <- sensor_tests_df$Wilcox_p_value < 0.05
table(sensor_tests_df$Significant)
```

```
##
## FALSE  TRUE
##    30    16
```

```r
head(sensor_tests_df, 10)  # top 10 sensors by significance
```

```
##     Sensor Wilcox_p_value Significant
## 59      59   1.201729e-10        TRUE
## 33      33   5.303166e-05        TRUE
## 129    129   7.213231e-05        TRUE
## 28      28   1.809467e-04        TRUE
## 316    316   6.390082e-04        TRUE
## 133    133   6.496350e-03        TRUE
## 155    155   7.225661e-03        TRUE
## 37      37   9.918747e-03        TRUE
## 417    417   1.169956e-02        TRUE
## 460    460   1.608327e-02        TRUE
```

After identifying the sensors with statistically significant differences in readings between passed and failed products, we narrowed our focus to just these 13 sensors. By filtering the data and visualizing these sensors with boxplots, we gained a clearer view of how their distributions differ by product outcome. This targeted approach helps emphasize the specific patterns—such as shifts in central tendency or increased spread—that may indicate early signs of product failure. These visual distinctions not only support the model's selection of these features but also offer practical insight into which parts of the production process might be contributing to defects.

```r
sig_sensors <- sensor_tests_df %>% filter(Significant) %>% pull(Sensor)

# Plot boxplots only for significant sensors
compare_long %>%
  filter(Sensor %in% sig_sensors) %>%
  ggplot(aes(x = Label, y = Reading, fill = Label)) +
  geom_boxplot() +
  facet_wrap(~ Sensor, scales = "free_y") +
  theme_minimal() +
```

```r
  labs(title = "Significant Sensor Readings by Product Outcome",
       y = "Sensor Reading",
       x = "Product Outcome")
```

The analysis of mean and median sensor readings across passed and failed products provides valuable insight into which features are most closely associated with product failure. Among the 13 statistically significant sensors, several exhibit meaningful shifts in their distributions that align with model-driven feature importance. For example, Sensor 59 shows a dramatic increase in mean readings for failed units, suggesting a strong relationship between elevated sensor values and product defects. Similarly, Sensors 129 and 163 display noticeable changes in their central tendencies, with Sensor 129 shifting from negative to near-zero values and Sensor 163 showing increased readings in failed units. These directional changes—both increases and decreases—highlight the nuanced nature of the signals tied to failure. By focusing on statistically validated sensors and comparing their distributions, we gain a clearer picture of the specific sensor behaviors that are likely contributing to the model's predictions, as well as a deeper understanding of the physical or operational conditions that may underlie product quality issues.

```r
summary_table <- compare_long %>%
  filter(Sensor %in% sig_sensors) %>%
  group_by(Sensor, Label) %>%
  summarise(
    Mean_Reading = mean(Reading, na.rm = TRUE),
    Median_Reading = median(Reading, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Sensor, Label)
```

```r
print(summary_table)
```

```
## # A tibble: 32 x 4
##    Sensor Label  Mean_Reading Median_Reading
##    <chr>  <fct>         <dbl>          <dbl>
##  1 126    Passed       2.75            2.74
##  2 126    Failed       2.81            2.78
##  3 129    Passed      -0.603          -0.142
##  4 129    Failed      -0.0887          0
##  5 133    Passed    1004.           1004.
##  6 133    Failed    1006.           1005.
##  7 14     Passed       8.95            8.86
##  8 14     Failed       8.24            8.50
##  9 155    Passed       0.513           0.32
## 10 155    Failed       0.398           0.38
## # i 22 more rows
```

Predictive Question: Can we build an accurate classification model (e.g., logistic regression) to predict whether a product will fail based on key sensor readings?

Yes, we can develop a classification model to predict product failure using key sensor readings, but there are several important considerations. Our analysis shows that certain sensors have statistically significant differences between passed and failed products, indicating these sensors carry useful predictive information. However, the dataset is imbalanced, with relatively few failure cases (about 5.6%), and the number of failure events relative to predictors is low, raising the risk of overfitting.

To address this, we employed regularized logistic regression (LASSO) which shrinks irrelevant variables, effectively performing feature selection and reducing overfitting risk. The model achieved an area under the ROC curve (AUC) of approximately 0.68 using the best tuning parameter, indicating moderate predictive ability.

While this performance suggests the model can discriminate failures better than random guessing, it is not highly accurate and may benefit from further refinement, such as incorporating more data, exploring alternative algorithms, or advanced imbalance handling techniques. Nonetheless, the results demonstrate the feasibility of predictive modeling in this context, providing a foundation for more robust failure prediction models in future work.

Overall, this was an enjoyable yet challenging project that highlights many common issues encountered with real-world datasets, especially those that violate standard modeling assumptions. It reinforced the importance of thorough data exploration and validation before diving into modeling. Developing a deep understanding of the models and their assumptions is critical for making appropriate choices and achieving effective, reliable predictions in practical applications. Real-world data rarely fits textbook scenarios, so careful, thoughtful analysis remains key to success.

Resources

Conceptual

Tibshirani, R. (1996). Regression shrinkage and selection via the LASSO. Journal of the Royal Statistical Society: Series B (Methodological), 58(1), 267–288.

Kuhn, M., & Johnson, K. (2019). Feature Engineering and Selection: A Practical Approach for Predictive Models. CRC Press.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning (with Applications in R). Springer.

Harrell, F. E. Jr. (2015). Regression Modeling Strategies: With Applications to Linear Models, Logistic and Ordinal Regression, and Survival Analysis. Springer.

Wickham, H., & Grolemund, G. (2016). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data. O'Reilly Media.

Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861-874.

Conover, W. J. (1999). Practical Nonparametric Statistics (3rd ed.). Wiley.

Coding

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L., François, R., … & Yutani, H. (2019). tidyverse: Easily install and load the 'tidyverse' (R package). https://www.tidyverse.org/

Kuhn, M. (2023). caret: Classification and Regression Training (R package). https://topepo.github.io/caret/index.html

Sing, T., Sander, O., Beerenwinkel, N., & Lengauer, T. (2005). ROCR: Visualizing classifier performance in R (R package). https://r-packages.io/packages/ROCR

Robin, X., Turck, N., Hainard, A., Tiberti, N., Lisacek, F., Sanchez, J. C., & Müller, M. (2011). pROC: Display and analyze ROC curves (R package version 1.18.5). https://www.rdocumentation.org/packages/pROC/versions/1.18.5

Friedman, J., Hastie, T., & Tibshirani, R. (2023). glmnet: Lasso and Elastic-Net Regularized Generalized Linear Models (R package). https://glmnet.stanford.edu/articles/glmnet.html

Kuhn, M., & Vaughan, D. (2022). themis: A Tidy Workflow for Handling Imbalanced Data (R package). https://themis.tidymodels.org/

Kuhn, M., & Vaughan, D. (2023). recipes: Preprocessing Tools to Create Design Matrices (R package). https://recipes.tidymodels.org/

Fox, J., & Weisberg, S. (2019). car: Companion to Applied Regression (R package). https://cran.r-project.org/web/packages/car/index.html

Andri Signorell et al. (2023). DescTools: Tools for Descriptive Statistics (R package). https://cran.r-project.org/web/packages/DescTools/index.html